Image Upload Pipeline

<details>
<summary>Relevant source files</summary>

- `Backend/STUDER-MEDIA/Dockerfile`
- `Backend/STUDER-MEDIA/main.py`
- `Backend/STUDER-MEDIA/placeholder.txt`
- `Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java`
- `Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java`
- `Backend/STUDER/src/main/java/facu/studer/config/AppConfig.java`
- `Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java`
- `Backend/STUDER/src/main/java/facu/studer/config/WebClientConfig.java`
</details>

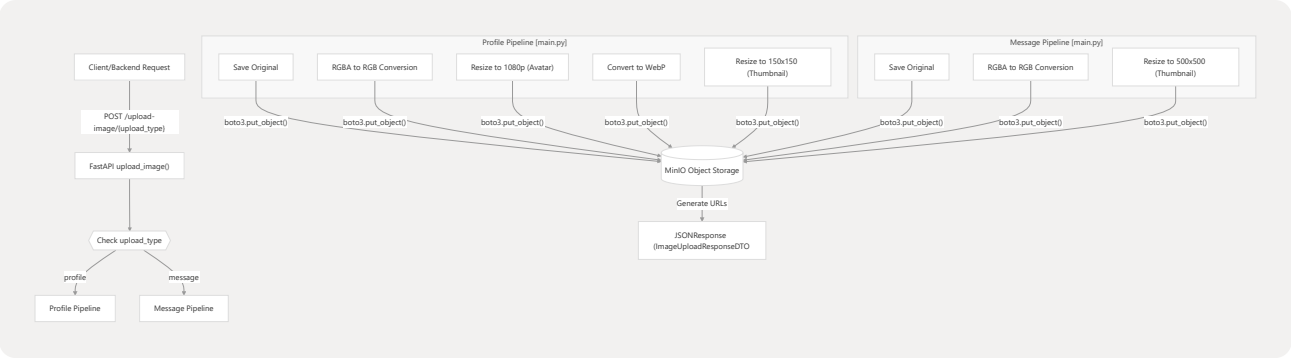
The Image Upload Pipeline is a specialized microservice flow within the **STUDER-MEDIA** service designed to handle the ingestion, transformation, and storage of binary image data. It provides a dedicated `POST /upload-image/{upload_type}` endpoint that abstracts complex image processing logic (resizing, format conversion, and variant generation) from the primary Spring Boot backend.

1. Pipeline Overview

The pipeline is triggered when the backend or a client sends a multipart file to the FastAPI application. The service distinguishes between two primary workflows based on the `upload_type` path parameter: `profile` and `message`.

Data Flow Diagram

This diagram illustrates the flow from the initial request to the final storage in MinIO and the response back to the caller.



Sources:

- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L97-L102" min=97 max=102 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Definition of the `upload_image` endpoint.
- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L111-L144" min=111 max=144 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Profile image processing logic.
- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L146-L162" min=146 max=162 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Message image processing logic.

2. Image Processing Logic

The service utilizes the **Pillow (PIL)** library to perform transformations. A critical step in both pipelines is the handling of transparency; since the system targets JPEG for several variants, it performs an explicit conversion from `RGBA` to `RGB` to prevent errors during save operations.

Transformation Variants

Variant	Dimensions	Format	Pipeline	Storage Path
Original	Source	Source	Both	`{type}/original/{filename}`
Avatar	1080x1080	JPEG	Profile	`profile/avatar/{filename}`
WebP	Source	WebP	Profile	`profile/webp/{filename}`
Thumbnail (Small)	150x150	JPEG	Profile	`profile/thumbnail/{filename}`
Thumbnail (Large)	500x500	JPEG	Message	`message/thumbnail/{filename}`

Code Implementation Details

- **RGBA Conversion:** Before resizing to JPEG, the service checks `img.mode`. If it is `RGBA`, it converts to `RGB` `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L119-L121" min=119 max=121 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`.
- **Thumbnailing:** The `img.thumbnail()` method is used to maintain aspect ratios while ensuring the image fits within the specified bounds `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L122-L138" min=122 max=138 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`.
- **In-Memory Buffering:** Images are processed in memory using `io.BytesIO` to avoid disk I/O overhead before being uploaded to MinIO `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L123-L125" min=123 max=125 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`.

Sources:

- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L117-L162" min=117 max=162 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Use of Pillow for resizing and format conversion.
- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L7-L8" min=7 max=8 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Import of `PIL.Image` and `io`.

3. Storage and Response

The service interacts with MinIO via the `boto3` S3 client. Upon successful upload of all variants, the service constructs a map of URLs.

Code-to-Entity Mapping

This diagram bridges the Python service logic with the Java DTOs that consume the data.

Response Structure

The endpoint returns a `JSONResponse` containing a dictionary of URLs. On the Java backend side, this is captured by the `ImageUploadResponseDTO`.

- **Python Response:** `return JSONResponse(content={"urls": urls})` `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L169-L169" min=169 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`.
- **Java DTO:** The `ImageUploadResponseDTO` contains a `Map<String, String> urls` to store these dynamic links `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java#L7-L9" min=7 max=9 file-path="Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java">Hi</FileRef>`.

- **User Mapping:** The `UserResponseDTO` in the Spring Boot application includes specific fields to hold these variant URLs for the frontend to consume `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java#L42-L48" min=42 max=48 file-path="Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java">Hii</FileRef>`.

Sources:

- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L169-L169" min=169 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Final response generation.
- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java#L1-L9" min=1 max=9 file-path="Backend/STUDER/src/main/java/facu/studer/DTOs/media/ImageUploadResponseDTO.java">Hii</FileRef>`: Backend DTO for media response.
- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java#L42-L48" min=42 max=48 file-path="Backend/STUDER/src/main/java/facu/studer/DTOs/user/UserResponseDTO.java">Hii</FileRef>`: User-specific image URL fields.

4. Error Handling and Logging

The pipeline includes comprehensive logging via a `RequestLoggingFilter` on the backend and standard Python logging in the media service.

- **Request ID Tracking:** Every request is assigned a short UUID to track its lifecycle across logs `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java#L25-L31" min=25 max=31 file-path="Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java">Hii</FileRef>`.
- **Media Service Logging:** The Python service logs the receipt of the file, its size, and the completion of each variant upload (e.g., "Uploaded WebP image") `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-`

`MEDIA/main.py#L109-L136" min=109 max=136 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`.

- **Validation:** If an unsupported `upload_type` is provided, the service returns a 400 Bad Request `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L164-L166" min=164 max=166 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`.

Sources:

- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java#L16-L49" min=16 max=49 file-path="Backend/STUDER/src/main/java/facu/studer/config/RequestLoggingFilter.java">Hii</FileRef>`: Backend request logging implementation.
- `<FileRef file-url="https://github.com/FacuRochaS/STUDER/blob/6e15e6ac/Backend/STUDER-MEDIA/main.py#L103-L172" min=103 max=172 file-path="Backend/STUDER-MEDIA/main.py">Hii</FileRef>`: Logging and error handling in the media service.